

FACULTAT D'INFORMÀTICA DE BARCELONA

TREBALL DE FINAL DE GRAU

Intèrpret Pas a Pas per a Programació Funcional

Genís Carretero Ferrete

Director: Gerard Escudero Bakx

Titulació: Grau en Enginyeria Informàtica (Computació)

Data: Maig 2024

Índex

I	Gestió del Projecte (GEP)	3
0.1	Introducció	4
0.2	Contextualització del Projecte	4
0.2.1	Marc del projecte i contextualització a la FIB	4
0.2.2	Definició de termes i conceptes clau	5
0.2.3	Identificació del problema a resoldre	6
0.2.4	Actors implicats i beneficiaris	6
0.2.5	Estat de l'art i alternatives existents	7
0.2.6	Justificació del projecte i valor afegit	8
0.3	Abast i Objectius del Projecte	9
0.3.1	Objectiu General	10
0.3.2	Objectius Específics	10
0.3.3	Límits del sistema i elements fora de l'abast	10
0.3.4	Requeriments Tècnics i Funcionals	11
0.3.5	Identificació de Riscos i Plans de Mitigació	12
0.4	Metodologia i Rigor de Desenvolupament	13
0.5	Planificació Temporal	15
0.5.1	Gestió del Calendari i Distribució d'Hores	15
0.5.2	Descripció Detallada de les Tasques, Precedències i Rols	15
0.5.3	Recursos Humans i Materials	16
0.5.4	Diagrama de Gantt	16
0.6	Pressupost i Sostenibilitat	18
0.6.1	Identificació i Justificació de Costos	18
0.6.2	Control de Gestió de Desviacions	19
0.6.3	Informe de Sostenibilitat	20
II	Desenvolupament Tècnic	21
1	Anàlisi Lèxica i Sintàctica	22
1.1	Disseny de la gramàtica	22
1.2	Implementació del Parser	23

1.2.1	Combinadors de Parsers	23
1.2.2	Gestió de Precedència i Associativitat	24
2	Sistema de Tipus	26
2.1	L'algorisme de Hindley-Milner	26
2.2	Unificació i Inferència	26
2.2.1	Variables de tipus fresques	26
2.3	Polimorfisme i Contextos	26
3	Avaluació i Reducció de Grafs	27
3.1	De l'AST al Graf	27
3.1.1	Estructura del Heap i Adreçament	27
3.2	Lambda Lifting i Supercombinadors	27
3.2.1	Extracció de variables lliures	27
3.2.2	Generació de funcions globals	27
3.3	Estratègia de Reducció (Lazy Evaluation)	27
4	Resultats i Proves	28
4.1	Bateria de Tests d'Inferència	28
4.2	Execució de Supercombinadors	28
4.3	Limitacions actuals	28
5	Conclusions	29
5.1	Objectius assolits	29
5.2	Aprenentatges personals	29
5.3	Treball futur	29

Part I

Gestió del Projecte (GEP)

0.1 Introducció

El present Projecte de Final de Grau (TFG) consisteix en el disseny, anàlisi i implementació d'un intèrpret educatiu per a un subconjunt del llenguatge de programació funcional Haskell. Aquesta eina neix amb el propòsit fonamental de resoldre la desconexió existent entre el model mental de l'alumne que s'inicia en el paradigma funcional i l'execució real que efectua la màquina.

A diferència dels llenguatges imperatius on l'estat de les variables i el flux de control s'alteren pas a pas, els llenguatges funcionals com Haskell operen mitjançant l'avaluació d'expressions de manera mandrosa (*lazy evaluation*). L'objectiu principal d'aquest projecte és obrir la "caixa negra" de l'avaluador funcional, creant una eina que permeti visualitzar l'arbre sintàctic abstracte i el procés de reducció d'expressions matemàtiques de forma interactiva, facilitant així la comprensió de conceptes complexos com la recursivitat pura, l'aplicació parcial de funcions i el filtratge per patrons (*pattern matching*).

0.2 Contextualització del Projecte

0.2.1 Marc del projecte i contextualització a la FIB

El present TFG s'emmarca dins de la intensificació en Computació del Grau en Enginyeria Informàtica impartit a la Facultat d'Informàtica de Barcelona (FIB) de la Universitat Politècnica de Catalunya (UPC). Concretament, el projecte està concebut per integrar-se de manera directa en les eines de suport docent de l'assignatura de Llenguatges de Programació (LP), una assignatura nuclear i obligatòria per a tots els estudiants del grau.

El pla d'estudis de la FIB està estructurat de manera que l'estudiant adquireix en els seus primers anys una formació molt sòlida en els paradigmes imperatiu i orientat a objectes, utilitzant llenguatges com C++ (a PRO1 i PRO2) i Java. Aquests paradigmes es basen en la mutabilitat de l'estat, els bucles estructurats (for, while) i una execució estrictament seqüencial. No obstant això, a l'assignatura de LP, els alumnes s'enfronten per primera vegada al paradigma funcional pur mitjançant el llenguatge Haskell.

Aquest salt cognitiu representa, històricament, una de les majors dificultats acadèmiques del grau. La programació funcional pura exigeix pensar en termes de transformació de dades i composició matemàtica en lloc d'instruccions de modificació de memòria. La natu-

ralesa mandrosa de Haskell agreuja aquesta transició, ja que l'ordre en què s'escriu el codi no correspon necessàriament a l'ordre en què la màquina l'avalua. Aquest TFG pretén proporcionar una eina docent directament aplicable als laboratoris de la FIB, alineant-se amb la competència transversal d'aprenentatge autònom i millorant significativament la ràtio de comprensió de la semàntica del llenguatge.

0.2.2 Definició de termes i conceptes clau

Per entendre l'abast teòric i la complexitat arquitectònica del projecte, és imprescindible definir amb rigor una sèrie de conceptes fonamentals propis de la teoria de compiladors i del paradigma funcional, que seran el nucli del motor a desenvolupar:

- **Programació Funcional Pura:** Un paradigma de programació declaratiu on els programes es construeixen exclusivament aplicant i component funcions matemàtiques pures. En aquest model no existeixen els efectes secundaris (*side-effects*) ni la mutabilitat de variables. Una funció, donats els mateixos arguments, retornarà sempre el mateix resultat.
- **Avaluació Mandrosa (Lazy Evaluation):** És una estratègia d'avaluació de computació que retarda el càlcul d'una expressió fins al moment en què el seu valor és estrictament necessari per a obtenir el resultat final del programa. Permet la creació i manipulació d'estructures de dades infinites i evita càlculs innecessaris.
- **Thunk:** Un encapsulament (sovint implementat com un punter a una funció i el seu entorn) que s'utilitza en l'avaluació mandrosa per representar una expressió no avaluada. L'interpret "força" *think* quan realment en necessita el valor.
- **Reducció de Grafs (Graph Reduction):** Mecanisme intern mitjançant el qual s'avaluen les expressions en Haskell. Les expressions es modelen com un graf dirigit acíclic en memòria. Els passos d'execució consisteixen a localitzar un subgraf avaluable (anomenat *redex*) i aplicar regles de reescriptura (beta-reduccions) fins a obtenir una forma normal.
- **Aplicació parcial (Currying):** Procés teòric pel qual una funció que rep múltiples arguments es transforma en una seqüència de funcions, cadascuna amb un sol argument. És la base del sistema de tipus funcional.
- **Filtratge per patrons (Pattern Matching):** Un mecanisme molt potent de control de flux i desestructuració de dades. Permet comprovar si una estructura complexa té una forma específica i enllaçar els seus components a variables d'àmbit local per al seu tractament.

- **Arbre Sintàctic Abstracte (AST - Abstract Syntax Tree):** Una representació en forma d'arbre de l'estructura lògica del codi font. Cada node de l'arbre denota un constructe del codi (una operació, una declaració, etc.). El parser serà l'encarregat de generar aquest AST a partir del text escrit per l'alumne.
- **Monadic Parser Combinators:** Una tècnica avançada de programació funcional per a l'anàlisi lèxica i sintàctica. Permet tractar els *parsers* (analitzadors) com a valors de primera classe. Mitjançant operacions monàdiques, es poden combinar petits analitzadors (ex. un que llegeix un dígit) per construir analitzadors de gramàtiques altament complexes (?).

0.2.3 Identificació del problema a resoldre

El problema troncal que motiva aquest TFG és la manca de visibilitat i retroalimentació formativa en el procés d'execució del codi funcional per a estudiants primerencs. Actualment, el procés d'aprenentatge es dona de la següent manera: un estudiant implementa un algorisme recursiu en l'interpret oficial (el Glasgow Haskell Compiler o GHCi), l'executa, i el sistema li retorna únicament el valor final. Si el resultat és erroni, o si s'esdevé un error com un bucle infinit o un *Stack Overflow*, l'interpret actua com una "caixa negra".

Aquesta opacitat significa que l'alumne no rep informació sobre *quina* substitució concreta ha provocat l'error o *com* ha evolucionat l'acumulador en una funció d'ordre superior com un `foldl` o `foldr`. La frustració derivada d'aquest fet allarga innecessàriament les hores de laboratori. La incapacitat d'observar com creix una expressió abans de ser reduïda fa que molts alumnes memoritzin patrons de codi en comptes d'interioritzar la semàntica del model de grafs. El problema a resoldre, per tant, és la manca d'un entorn pedagògic visual que "alenteixi la màquina i forci la transparència pas a pas, prioritant la claredat educativa envers el rendiment d'execució.

0.2.4 Actors implicats i beneficiaris

S'han identificat i caracteritzat els següents perfils relacionats directament o indirectament amb el cicle de vida del projecte:

1. **Estudiants universitaris (Beneficiaris directes):** Els alumnes de l'assignatura LP de la FIB i qualsevol persona que s'estigui introduint en Haskell. Són els usuaris finals de l'aplicació. Utilitzaran l'interpret per verificar hipòtesis de funcio-

nament, entendre els errors del seu codi i assimilar conceptes abstractes observant gràficament les transformacions del codi pas a pas.

2. **Equip Docent i Professorat (Facilitadors i beneficiaris indirectes):** Els professors de laboratori i de teoria de la FIB. El professorat disposarà d'una eina per il·lustrar de manera dinàmica exemples complexos a la pissarra o projector. Podran crear exercicis on els estudiants hagin de predir el "següent pas" de reducció abans de fer clic al botó de l'interpret.
3. **Institució (La Facultat i el Departament de CS):** La FIB es beneficia en millorar les seves ràtios d'aprovats i de satisfacció a les assignatures de programació avançada, dotant-se d'eines docents de creació pròpia.
4. **Desenvolupador i Investigador (Jo):** Com a autor d'aquest TFG, actuo com a analista de requeriments, arquitecte de programari i desenvolupador, aprofundint els coneixements en la construcció de compiladors, l'enginyeria de programari àgil i el disseny de la interacció usuari.

0.2.5 Estat de l'art i alternatives existents

L'ecosistema d'eines al voltant del desenvolupament per a Haskell és profund i madur, però està fortament orientat a l'entorn industrial i a enginyers de programari sèniors. Una anàlisi de les eines existents permet evidenciar per què s'ha de dissenyar una solució completament nova des de zero per a l'àmbit docent.

- **GHCi Debugger (oficial):** És el depurador inclòs de sèrie amb GHC. Tot i permetre introduir punts d'aturada (*breakpoints*) i inspeccionar variables, el seu ús és exclusivament per línia de comandes textual. El seu gran defecte educatiu és que la informació que bolca correspon a l'estat intern del compilador (conegut com a Haskell Core), on el codi ja ha patit nombroses optimitzacions i *desugarings* (eliminació de "sucre sintàctic"), sent sovint incompreensible respecte al codi original que l'alumne havia escrit.
- **Eines acadèmiques de visualització de grafs (GHood i Haskell Visualizer):** Durant els anys 2000 es van publicar algunes tesis i aplicacions (com GHood) per visualitzar el graf d'avaluació de Haskell en programes amb Java. El problema és que estan basades en versions antiquades de la sintaxi, requereixen processos complexos de compilació (no són **plug-and-play**), i només mostren una topologia d'arbre estàtica un cop el programa ja ha finalitzat, en lloc de permetre a l'alumne prémer "següent" per veure el canvi immediat a la línia de codi.

- **Hoed:** Una eina moderna de depuració algorísmica per a Haskell. És molt potent però segueix enfocada a usuaris avançats. Utilitza tècniques de instrumentació de codi invasives (l'usuari ha de posar etiquetes `observe` per tot el codi font per poder veure el seu valor). Això requereix aprendre una eina per poder aprendre el llenguatge, afegint fricció.
- **Python Tutor (L'Estàndard Docent de Referència):** Aquesta eina web creada per Philip Guo és el referent mundial absolut en docència de programació. Permet veure el codi Python, Java o C a l'esquerra i la pila de crides i punters a la dreta pas a pas. No obstant això, Python Tutor no suporta cap llenguatge funcional pur. La seva arquitectura està dissenyada per a l'avaluació estricta i els mapes de memòria imperatius, la qual cosa fa impossible adaptar-la directament a un sistema de reducció de grafs d'avaluació mandrosa.

L'absència d'un "Haskell Tutor" equivalent és el buit exacte que aquest TFG pretén omplir.

0.2.6 Justificació del projecte i valor afegit

Per tal de justificar de manera diàfana i esquemàtica el valor d'aquesta solució envers el que ja ofereix el mercat, es presenta la següent taula comparativa detallada:

Taula 1: Comparativa de la solució proposada respecte a les alternatives actuals

Característica	Avaluada	El Nostre Intèrpret (TFG)	GHCi	Py. Tutor
Públic objectiu principal		Estudiants i professors (Àmbit Acadèmic)	Enginyers/Sèniors	Principiants (Acadèmic)
Llenguatge suportat		Subconjunt docent de Haskell (FIB)	Haskell complet	Python/C/Java
Visualització gràfica del procés d'avaluació		Sí (Text ressaltat pas a pas i AST)	No (Consola textual incomprensible)	Sí (Variables en memòria)
Control temporal de l'execució		Navegació interactiva endavant i endarrere	Només salt endavant entre punts d'aturada	Endavant i endarrere
Requisits d'Instal·lació		Cap (Accés via web o executable portable)	Alta (Requereix GHC, Cabal o Stack)	Cap (Navegador)
Tractament explícit de l'avaluació mandrosa		Representació gràfica de <i>Thunks</i> pendents	Amagat internament	N/A (És estricte)

A més de l'evident salt en usabilitat que s'observa a la taula, a nivell arquitectònic l'aportació clau és la utilització del paradigma de **Monadic Parser Combinators** (??) per al desenvolupament del nucli (*core*). Aquesta metodologia, en detriment d'usar eines automàtiques de generació de parsers com Lex/Yacc o Alex/Happy, proporciona dos beneficis fonamentals: primer, el codi de l'interpret s'escriu completament en Haskell natiu; i segon, la definició del parser s'assembla pràcticament línia a línia a la gramàtica BNF oficial. Això és decisiu perquè qualsevol professor de la FIB, en el futur, pugui obrir el codi d'aquest TFG i afegir una nova funcionalitat sintàctica sense haver de ser un expert en eines de tercers. L'arquitectura garantirà, a més, la capacitat de generar missatges d'error personalitzats fets a mida per l'assignatura.

0.3 Abast i Objectius del Projecte

L'establiment rigorós de l'abast és vital per no incorrer en una expansió incontrolada de les prestacions del programari (*scope creep*), donada la immensitat del llenguatge Haskell.

0.3.1 Objectiu General

L'objectiu central i global d'aquest projecte és **dissenyar, programar, testar i validar** un intèrpret educatiu funcional per a un subconjunt controlat del llenguatge Haskell, dotat amb una interfície d'usuari (UI) interactiva que exposi al detall cada pas computacional de la semàntica de reducció, convertint l'execució en un procés transparent i formatiu.

0.3.2 Objectius Específics

Per assolir amb èxit l'objectiu general, s'estableixen els següents sub-objectius seqüencials:

- **Definir** formalment els límits del subconjunt del llenguatge Haskell que l'eina serà capaç de suportar. Això inclourà tipus bàsics (enters, booleans), llistes simples, definicions de funcions mitjançant equacions, estructures condicionals (**if-then-else**), aplicació parcial i funcions d'ordre superior clàssiques (**map**, **filter**, **fold**).
- **Construir** l'analitzador lèxic (que tokenitzi l'entrada) i l'analitzador sintàctic emprant la llibreria *Parsec* o *Megaparsec*, assegurant una alta tolerància i amabilitat en la reportació d'errors lèxics de l'usuari.
- **Implementar** el motor de reducció basat en un entorn (*Environment*), dissenyant un sistema d'avaluació estructurada que pugui interrompre l'execució a cada aplicació delta o beta, guardar-ne l'estat en memòria i continuar sota demanda seguint la regla "lazy".
- **Desenvolupar** una Interfície Gràfica d'Usuari (GUI) intuïtiva, minimalista i robusta on els estudiants puguin enganxar el seu codi al panell esquerre i navegar per una traça temporal visual d'execució al panell dret.
- **Validar** el funcionament correcte i la idoneïtat pedagògica del conjunt de l'eina, realitzant proves sobre bateries de problemes reals trets directament dels exàmens i pràctiques històriques de l'assignatura de LP de la FIB.

0.3.3 Límits del sistema i elements fora de l'abast

Tan important com saber què s'ha de fer és tenir clar allò que **no** està inclòs per mantenir el projecte dins els marges de viabilitat de temps (540 hores). Queden exclosos explícitament els següents aspectes de Haskell:

- El sistema complet d'inferència de tipus (tipus Hindley-Milner). L'interpret assumirà que el codi introduït és semànticament correcte i tipat fort, confiant en la reducció mecànica. No es desenvoluparà un *Type Checker* estricte prèvi.
- Operacions d'Entrada/Sortida (I/O). No s'implementarà la mónada IO ni funcions com `putStrLn` o `readFile`, atès que el projecte se centra en expressions pures matemàtiques.
- Suport avançat a les Classes de Tipus (*Typeclasses*) i instanciació de variables complexes de context.
- Mòduls, importacions d'arxius externs o el paquet base de biblioteques estàndard (a excepció de les operacions aritmètiques primitives pre-carregades a l'entorn).

0.3.4 Requeriments Tècnics i Funcionals

Dels objectius anteriors es destil·len els requeriments del producte de programari:

Requeriments Funcionals (RF):

- **RF1 (Entrada):** L'aplicació ha de disposar d'un panell d'edició on l'usuari pugui introduir el codi font en format de text lliure, amb ressaltat de sintaxi Haskell.
- **RF2 (Parsing i Gestió d'Errors):** En sol·licitar l'execució, l'interpret ha de transformar el text en un AST i aturar-se de forma immediata mostrant la línia i columna exacta on l'usuari hagi comès un error sintàctic.
- **RF3 (Motor de Traça):** El sistema ha de calcular i retenir en estructures de dades seqüencials tots els estats intermedis del codi, emulant els passos lògics d'un ésser humà reduint amb llapis i paper.
- **RF4 (Control):** S'ha d'oferir a l'usuari una botonera estil reproductor multimèdia: Reproduir cap al final, Següent Pas, Pas Anterior, i Tornar a l'inici.
- **RF5 (Ressaltat de Redex):** A cada instant de la traça visual, l'aplicació ha de marcar (per exemple amb un color diferent o en negreta) quina sub-expressió exacta s'acaba d'avaluar o quina s'avaluarà immediatament després (el *redex*).

Requeriments No Funcionals (RNF):

- **RNF1 (Usabilitat - Zero Fricció):** Seguint les heurístiques de Nielsen, l'eina ha de garantir que un usuari assoleixi executar el seu primer codi amb un màxim de

dos clics des de l'inici de la sessió. Ha de requerir zero descàrregues o instal·lacions (prioritzant la tecnologia d'entorn d'aplicació web SPA).

- **RNF2 (Eficiència Temporal):** Tot i no estar optimitzat per a grans processos corporatius, l'anàlisi previ del codi docent no ha de superar un llindar màxim d'espera de 3 segons per evitar pèrdua d'atenció per part de l'estudiant.
- **RNF3 (Mantenibilitat):** El codi font estarà ben documentat (usant *Haddock* o documentació en línia equivalent) facilitant-ne futures extensions al departament de CS de la FIB.

0.3.5 Identificació de Riscos i Plans de Mitigació

L'enginyeria d'un compilador docent comporta un seguit de riscos tècnics i conceptuals d'alt impacte que requereixen d'estratègies preventives adequades:

- **Risc 1: Explosió combinatòria de la memòria per recursivitat infinita.**
 - *Explicació:* Estudiants introduint bucles infinits sense condició de parada col·lapsaran la memòria RAM de l'eina intentant guardar els passos històrics.
 - *Mitigació (Pla B):* Establir per defecte un tall de seguretat forçat (per exemple, aturar la reducció asíncrona si s'arriba a 1.000 passos) avisant l'usuari amb un missatge clar de "Possibilitat de recursivitat infinita detectada".
- **Risc 2: Dificultat i lentitud en la instrumentació de l'avaluació mandrosa.**
 - *Explicació:* Gestionar estats immutables i clausures (*closures*) mentre s'extreu gràficament la informació pot presentar una dificultat superior al model imperatiu inicialment previst.
 - *Mitigació (Pla B):* Si el disseny del motor de grafs resulta inabastable als dos mesos de desenvolupament, es reduirà l'abast del projecte per suportar inicialment només l'avaluació aplicativa (estRICTA) limitant funcions infinites, però assegurant en qualsevol cas el lliurament d'un intèrpret funcional i educatiu.
- **Risc 3: Ambigüitats en l'anàlisi gramatical (Parsing).**
 - *Explicació:* La gramàtica de Haskell és famosa per algunes particularitats estructurals com la regla d'identació significativa (*off-side rule*) o la complexitat de l'aplicació infixa d'operadors.

- *Mitigació (Pla B)*: Eliminar l'anàlisi basada en identació i requerir de manera obligatòria claus explícites i punts i coma { ; } per a la versió inicial de l'interpret docent, fet que simplifica enormement l'AST generat.

0.4 Metodologia i Rigor de Desenvolupament

L'elevada incertesa de dissenyar una aplicació altament tècnica fa inviable l'ús de metodologies clàssiques o predictives (model en Cascada). En conseqüència, es seguirà una **metodologia iterativa i incremental** amb un fort enfocament àgil basat en les cerimònies i pràctiques del **marc de treball Scrum (?)**. Malgrat ser un projecte individual, l'alumne aplicarà aquest rigor assumint de facto els rols fonamentals: actuarà com a *Product Owner* definint els requeriments i alhora com a equip de desenvolupament de manera autoorganitzada.

Estructura de Cicles Iteratius (Sprints): El projecte s'estructurarà en cicles anomenats *Sprints* d'una durada constant de 2 a 3 setmanes. Al finalitzar cadascun d'aquests Sprints s'assolirà una fita rellevant; la generació d'un Increment del Producte potencialment entregable. La lògica evolutiva de l'increment serà:

- *Sprint 1*: Analitzador que suporta operacions enteres i literals.
- *Sprint 2*: Suport a declaració i aplicació de funcions simples.
- *Sprint 3*: Gestió i expansió de llistes i *Pattern Matching*.
- *Sprint 4*: Motor iteratiu complet amb historial de traça.
- *Sprints 5-6*: Frontend gràfic, resolució de deute tècnic i optimització.

Aquest model basat en el concepte de Producte Mínim Viable (MVP) contínu permet adaptar la direcció tècnica o retallar funcionalitats perifèriques depenent del temps real dedicat, sense comprometre en cap cas la viabilitat nuclear de l'entrega final.

Eines, Protocols i Control Qualitatiu:

- **Gestió del Flux (Tauler Kanban)**: S'emprarà un tauler a Jira o Trello per centralitzar l'estat del projecte. Les funcions principals es redactaran com a Històries d'Usuari i es traslladaran successivament per les columnes *Product Backlog*, *To Do*, *In Progress*, *Testing*, i *Done*.

- **Control de Versions Sòlid (Git i GitHub):** Tot el desenvolupament de codi es traçarà amb el sistema Git de repositoris històrics. S'utilitzarà el protocol *Feature Branch Workflow*, on cada nova funcionalitat es codificarà en una branca independent abans de ser fusionada a la principal mitjançant *Pull Requests*. Això és una garantia d'integritat del codi.
- **Polítiques de Validació Contínua (TDD i BDD):** Com a garantia de fiabilitat del core computacional del compilador, la pràctica de proves s'executarà simultàniament a la creació usant el mètode de *Test-Driven Development*. Eines com *HUnit* permetran escriure una prova que verifiqui el format d'avaluació correcta segons les normes de GHC abans d'implementar el detall de l'algorisme monàdic corresponent.
- **Cerimònies de Validació Externa (Sprint Reviews):** Cada quinzenal es realitzarà una reunió presencial o telemàtica amb el Director de Projecte de la FIB. En aquestes tutories s'ensenyarà exclusivament programari funcionant, es discutirà el *feedback* pedagògic de la interfície generada i es consensuarà l'enfocament i els possibles plans de mitigació del Sprint vinent. Aquest *loop* de realimentació és crucial per alinear el projecte amb el pla educatiu establert de forma pragmàtica.

0.5 Planificació Temporal

0.5.1 Gestió del Calendari i Distribució d'Hores

El TFG de la Facultat d'Informàtica de Barcelona consta de 18 crèdits ECTS. Assumint una dedicació de 30 hores per crèdit, el projecte té una **durada total de 540 hores de dedicació de l'estudiant**. El projecte s'executa oficialment des del **15 de febrer de 2026 fins al 15 de juny de 2026** (17 setmanes totals d'execució, on l'última setmana es destina exclusivament a la defensa). Per assumir les 540 hores, la distribució de dedicació de l'estudiant serà d'aproximadament 32 hores setmanals de mitjana, que s'ajustaran depenent de la intensitat de la fase (major dedicació durant la implementació del nucli tecnològic).

0.5.2 Descripció Detallada de les Tasques, Precedències i Rols

S'ha abandonat l'agrupació per fases genèriques per detallar un a un els elements atòmics de treball. S'assignen diferents "rols" a l'estudiant per facilitar posteriorment la imputació econòmica.

Taula 2: Taula detallada de tasques, durada, precedències i rols associats

ID	Nom de la Tasca	Hores	Preced.	Rol Assignat	Recurs Material
T1	Gestió del Projecte (GEP)	110h			
T1.1	Lliuraments i Context GEP	30	-	Gestor Proj.	Biblio/Office
T1.2	Planificació i Gantt	20	T1.1	Gestor Proj.	LaTeX
T1.3	Pressupost i Sostenibilitat	20	T1.2	Gestor Proj.	Excel
T1.4	Redacció Memòria TFG	40	T1.1	Gestor Proj.	LaTeX
T2	Recerca i Base Teòrica	65h			
T2.1	Estudi Semàntica i GHC	35	-	Eng. Programari	Documents
T2.2	Investigació Parsers Monàdics	30	-	Eng. Programari	PC
T3	Disseny Arquitectònic	65h			
T3.1	Disseny Gramàtica i AST	35	T2.1	Eng. Programari	PC
T3.2	Mockups GUI	30	-	Eng. Programari	Figma

ID	Nom de la Tasca	Hores	Preced.	Rol Assignat	Recurs Material
T4	Implementació Core	160h			
T4.1	Desenvolupament Lexer	30	T3.1	Eng. Programari	GHC, Git
T4.2	Desenvolupament Parser	40	T4.1	Eng. Programari	GHC, Git
T4.3	Motor de Reducció Mandro- sa	60	T4.2	Eng. Programari	GHC, Git
T4.4	Captura i Historial d'Estats	30	T4.3	Eng. Programari	GHC, Git
T5	Frontend i Qualitat	140h			
T5.1	Implementació UI	50	T3.2	Eng. Programari	IDE Web
T5.2	Connexió UI - Haskell Mo- tor	40	T4.4,T5.1	Eng. Programari	IDE Web
T5.3	Bateria Proves Unitàries	30	T4.3	Analista QA	HUnit
T5.4	Proves Integració i Docs	20	T5.2	Analista QA	Git
T6	Defensa Pública	Total	540h		
T6.1	Preparació material Defen- sa	0h	T5.4	Gestor Proj.	Ppoint

0.5.3 Recursos Humans i Materials

La realització de les tasques prèvies implica:

- **Recursos Humans (RRHH):** Es defineixen 3 rols principals que assumeix l'estudiant: *Gestor de Projectes* (responsable de GEP i Memòria), *Enginyer de Programari* (recerca, disseny i programació) i *Analista de QA* (proves). A més, el *Director del TFG* actua com a validador de qualitat invertint unes 15 hores globals en reunions de seguiment.
- **Recursos Materials:** Ordinador portàtil personal, compilador de Haskell (GHC), gestor de paquets Stack, plataformes de repositoris (GitHub), programari ofimàtic i de disseny gràfic per a mockups (Figma) i llibreries de codi obert gratuïtes.

0.5.4 Diagrama de Gantt

S'ha modelat el diagrama de Gantt respectant les dependències marcades a la taula anterior, assegurant que les activitats de recerca i documentació es solapen de manera

concurrent i realista amb les de desenvolupament, atès que l'estudiant distribueix la seva jornada.

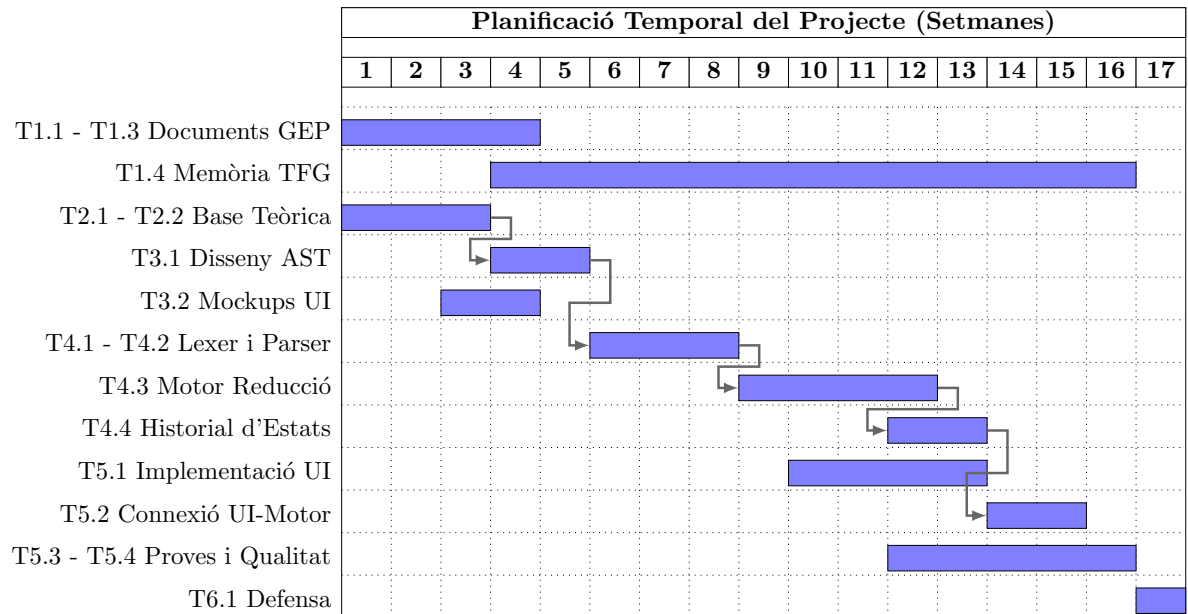


Figura 1: Diagrama de Gantt detallat amb concurrències i la ruta crítica d'implementació.

0.6 Pressupost i Sostenibilitat

Aquest apartat quantifica la viabilitat econòmica del projecte assignant valors als rols definits en la planificació temporal. També s'estableix un control de desviacions rigorós i s'avaluen les tres dimensions de sostenibilitat exigides.

0.6.1 Identificació i Justificació de Costos

Per dissenyar el pressupost d'una manera fidel a un entorn empresarial s'imputen quatre tipus de costos: Costos directes de personal, Amortitzacions de material, Despeses generals i Fons de seguretat (contingències i imprevistos).

Costos Directes de Personal per Tasca

Tot i ser un projecte acadèmic de l'alumne, se simulen els costos laborals. Els salaris per hora s'han justificat utilitzant dades de l'Acord Marc del sector TIC d'Espanya i portals com Glassdoor (dades referenciades per a l'any en curs):

- **Gestor de Projectes (Project Manager):** 35 €/hora. Gestiona documentació i fites.
- **Enginyer de Programari (Software Engineer):** 25 €/hora. Disseny i picar codi.
- **Analista de Qualitat (QA):** 20 €/hora. Creació de tests i validació d'errors.

A continuació es detalla el cost específic per cada activitat basat en la Taula 2:

Taula 3: Costos de Personal per Tasques del Gantt

Tasca	Rol Responsable	Hores	Cost Total (€)
T1. Gestió del Projecte (GEP)	Gestor de Projectes	110h	3.850,00 €
T2. Recerca i Base Teòrica	Eng. Programari	65h	1.625,00 €
T3. Disseny Arquitectònic	Eng. Programari	65h	1.625,00 €
T4. Implementació Core	Eng. Programari	160h	4.000,00 €
T5.1 i T5.2 (Front i Connexió)	Eng. Programari	90h	2.250,00 €
T5.3 i T5.4 (Proves QA)	Analista de Qualitat	50h	1.000,00 €
Total Personal directe	-	540h	14.350,00 €

Costos Generals i Amortitzacions

Per al hardware, s'estima una vida útil de l'ordinador portàtil de treball de 4 anys (48 mesos). Sent el seu valor de 2.000€ i utilitzant-se durant els 4 mesos del projecte, l'amortització és de **166,66 €**. Tot el programari utilitzat és lliure i de cost zero. Les despeses indirectes d'oficina (Internet i consum elèctric de l'equip) s'estimen en una tarifa plana imputada de **100,00 €** per tot el cicle.

Estimació del Pressupost Total

S'aplica un 15% de Fons de Contingència per cobrir riscos documentats al Gantt i un 5% d'imprevistos fora del control de l'abast.

Taula 4: Pressupost Global i Totalitzat

Concepte (Partida)	Cost Estimad
Total Costos de Personal	14.350,00 €
Amortització Maquinari	166,66 €
Despeses Generals (Indirectes)	100,00 €
Suma Base del Projecte	14.616,66 €
Contingències (15%)	2.192,50 €
Imprevistos (5%)	730,83 €
PRESSUPOST FINAL TOTAL	17.539,99 €

0.6.2 Control de Gestió de Desviacions

Amb l'objectiu d'assegurar l'excel·lència operativa i evitar desviacions econòmiques respecte a l'estimació anterior, s'implementarà la tècnica **EVM (Earned Value Management)**. Aquesta tècnica es monitoritzarà quinzenalment. S'utilitzaran tres mètriques base:

- **Valor Planificat (PV):** Cost estimat de les tasques que, segons el Gantt, haurien d'estar finalitzades.
- **Cost Real (AC):** Hores invertides realment per l'estudiant multiplicades pel salari del seu rol actiu.
- **Valor Guanyat (EV):** Cost original pressupostat de la part de les tasques que realment s'ha completat fins avui.

Els indicadors numèrics facilitadors del control seran:

- **SV (Variació del Calendari) = EV - PV.** Un valor inferior a 0 indica retard sobre el Gantt.
- **CV (Variació del Cost) = EV - AC.** Un valor inferior a 0 indica sobrecost econòmic.

Si l'índex CPI (EV/AC) cau per sota de l'1.0 durant les fases centrals, es destinarà partida del Fons de Contingència i, de ser necessari, es cancel·laran requeriments no funcionals secundaris per retornar al pressupost d'equilibri.

0.6.3 Informe de Sostenibilitat

Abans de la redacció d'aquesta memòria s'ha completat la matriu i l'enquesta d'autoavaluació de Sostenibilitat EDINSOST requerida per la facultat. Les conclusions per a les 3 dimensions són:

- **Dimensió Econòmica:** És un projecte altament viable. El seu desenvolupament i explotació es realitza amb programari completament de codi obert i no requereix servidors d'alt rendiment per ser allotjat, cosa que eximeix la universitat o l'usuari final de pagar cap tipus de llicència o subscripció futura.
- **Dimensió Ambiental:** La petjada ecològica és mínima i està circumscrita únicament als 32 kWh (aproximadament) consumits per l'ordinador portàtil de l'estudiant durant les 540 hores d'execució.
- **Dimensió Social:** El component d'impacte social és enorme. Millora radicalment l'entorn docent, afavoreix la inclusió acadèmica disminuint l'elevada corba d'aprenentatge del paradigma funcional i prevé l'abandonament o la frustració d'estudiants. La futura publicació del codi de forma lliure fomenta el coneixement compartit globalment.

Part II

Desenvolupament Tècnic

Capítol 1

Anàlisi Lèxica i Sintàctica

L'anàlisi d'un programa font es divideix tradicionalment en dues fases: l'anàlisi lèxica i l'anàlisi sintàctica. Aquest intèrpret s'allunya d'eines externes generadores de codi com *Happy* o *Alex* i opta per un enfocament natiu basat en **combinadors de parsers** encastats directament al llenguatge de programació (Haskell). Això permet mantenir l'homogeneïtat del codi i simplifica el disseny.

Per tal d'il·lustrar de manera didàctica el funcionament intern de l'intèrpret, aquesta memòria utilitzarà una **expressió guia** que ens acompanyarà al llarg de tots els capítols. Veurem el "viatge" complet d'aquesta expressió, des que és text pla fins que s'avalua. L'expressió triada és:

$$(\backslash x\ y \rightarrow x * y)\ 2\ 3$$

En aquest primer capítol, veurem com aquesta cadena de text es transforma en una estructura de dades comprensible per a la màquina.

1.1 Disseny de la gramàtica

L'objectiu principal és construir una representació abstracta en forma d'arbre (AST, *Abstract Syntax Tree*) de qualsevol expressió introduïda per l'usuari. La gramàtica suportada s'ha dissenyat inspirant-se fortament en un subconjunt fonamental del llenguatge Haskell i del Càlcul Lambda clàssic, seguint el model exposat per ?.

La nostra expressió guia conté gairebé tots els elements essencials que el sistema reconeix:

- **Càlcul Lambda:** Definicions anònimes de funcions mitjançant la sintaxi convencional (`\x y -> ...`).
- **Valors i Variables:** Nombres enters com el 2 i el 3, i variables com la `x` i la `y`.
- **Operadors Aritmètics:** Multiplicació (`*`), a més de suportar addició (`+`), subtracció (`-`) i divisió (`/`).
- **Aplicació de Funcions:** L'acció de passar el 2 i el 3 a la funció, establerta simplement per juxtaposició (espais entre elements).

A més d'aquests, el sistema també reconeix operadors relacionals (`<`, `==`, etc.) i lògics (`||`, `&&`), així com l'ignoratge de comentaris.

1.2 Implementació del Parser

1.2.1 Combinadors de Parsers

Un parser en Haskell pot entendre's com una funció que pren una cadena de text, intenta consumir-ne un fragment, i retorna una llista de parells amb el valor analitzat i la part restant de la cadena.

```
1 newtype Parser a = Parser {parse :: String -> [(a, String)]}
```

Listing 1.1: Definició monàdica del tipus Parser

Aquesta representació adquireix la seva màxima potència quan s'instancien les classes de tipus `Functor`, `Applicative`, `Monad` i `Alternative`. Aquesta arquitectura permet encadenar petits parsers lèxics (com `sat`, `digit` o `lletra`) per formar patrons més complexos. Un exemple clar és la combinació monàdica per analitzar una seqüència repetida mitjançant l'operador d'alternança `<|>`:

```
1 mul :: Parser a -> Parser [a]
2 mul p = do
3   x <- p
4   xs <- mul p
5   return (x : xs)
6   <|> return []
7
8 mes :: Parser a -> Parser [a]
9 mes p = do x <- p; xs <- mul p; return (x : xs)
```

Listing 1.2: Combinadors de repetició `mul` i `mes`

Gràcies a aquesta construcció *bottom-up*, les diferents parts de la nostra expressió (com el **token** del parèntesi o l'anàlisi dels enters 2 i 3) es resolen amb una concisió extrema i tolerant espais en blanc.

1.2.2 Gestió de Precedència i Associativitat

El repte més complex de l'anàlisi sintàctica rau en l'ambigüitat natural del llenguatge funcional. Calia dissenyar un parser que atorgués l'associativitat i prioritat correcta a les operacions, convertint la seqüència lineal en un arbre estructurat.

Per garantir la correcta **precedència d'operadors**, s'ha organitzat l'anàlisi de forma jeràrquica, de menys a més precedència: lògics, relacionals, sumatoris, multiplicatius i, per sobre de tots, l'aplicació de funcions. Així, a l'expressió $x * y$, l'operador multiplicatiu s'avalua abans que qualsevol suma que hi pogués haver al voltant.

Aplicació de Funcions i Currificació

El concepte més crític de la nostra gramàtica és l'aplicació de funcions. En programació funcional, l'aplicació és **associativa per l'esquerra**. La decisió de dissenyar el parser amb aquesta propietat no és arbitrària, sinó que és l'encarnació sintàctica de la **currificació** (*currying*).

En el nostre llenguatge, l'expressió guia $(\lambda x y \rightarrow x * y) 2 3$ no està passant dos arguments simultanis a una funció. El parser ho agrupa implícitament com a $((\lambda x y \rightarrow x * y) 2) 3$. Primer s'aplica el 2, retornant una nova funció que accepta la y , i finalment se li aplica el 3. S'ha resolt de manera eficient extraient el primer àtom i aplicant una reducció per l'esquerra a la resta d'arguments:

```
1 ap :: Parser Expr
2 ap = do
3     x <- atom
4     ys <- mul atom
5     return (insert (x:ys))
6     where
7         insert :: [Expr] -> Expr
8         insert [e] = e
9         insert exprs = (App (insert (init exprs)) (last exprs))
```

Listing 1.3: Associativitat esquerra per a l'aplicació

Un cop el parser ha processat completament la nostra cadena de text, genera el següent

Arbre de Sintaxi Abstracta (AST), que és l'objectiu final d'aquest capítol i el punt de partida per a la inferència de tipus:

Un cop el parser ha processat completament la nostra cadena de text, genera l'Arbre de Sintaxi Abstracta (AST). Com es pot observar a la Figura 1.1, la currificació es fa evident en veure com les aplicacions (**App**) s'apilen associant-se cap a l'esquerra:

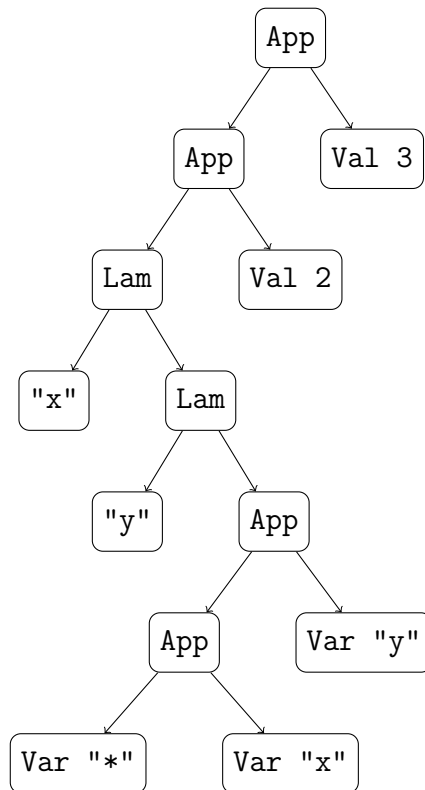


Figura 1.1: Arbre de Sintaxi Abstracta (AST) per a l'expressió $(\lambda x y \rightarrow x * y) 2 3$

Capítol 2

Sistema de Tipus

2.1 L'algorisme de Hindley-Milner

2.2 Unificació i Inferència

2.2.1 Variables de tipus fresques

2.3 Polimorfisme i Contextos

Capítol 3

Avaluació i Reducció de Grafs

3.1 De l'AST al Graf

3.1.1 Estructura del Heap i Adreçament

3.2 Lambda Lifting i Supercombinadors

3.2.1 Extracció de variables lliures

3.2.2 Generació de funcions globals

3.3 Estratègia de Reducció (Lazy Evaluation)

Capítol 4

Resultats i Proves

4.1 Bateria de Tests d'Inferència

4.2 Execució de Supercombinadors

4.3 Limitacions actuals

Capítol 5

Conclusions

5.1 Objectius assolits

5.2 Aprenentatges personals

5.3 Treball futur